

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH**



THIẾT KẾ HỆ THỐNG SỐ VỚI HDL

**BÁO CÁO ĐỒ ÁN
THIẾT KẾ HỆ THỐNG MÃ HÓA RSA**

GVHD: Th.s Ngô Hiếu Trường

Sinh viên thực hiện:

Nguyễn Đình Huy	-	23520624
Nguyễn Huỳnh Quốc Huy	-	23520633
Bùi Hoàng Anh Huy	-	23520592

Tp. Hồ Chí Minh, 21/05/2025

Mục lục : [GitHub Source DoAN](#)

A. GIỚI THIỆU ĐỀ TÀI:	3
1. Đặt vấn đề	3
2. Mục tiêu nghiên cứu	3
3. Đối tượng và phạm vi nghiên cứu	4
4. Cấu trúc đồ án	5
B. CƠ SỞ LÝ THUYẾT VÀ THUẬT TOÁN RSA	6
1. Tổng quan về mã hóa bất đối xứng	6
2. Cơ sở toán học của RSA	6
2.1 Phép toán số học mô-đun và lũy thừa hàm mũ (Modular Arithmetic)	6
2.2 Tập hợp số nguyên tố và bài toán nhân số nguyên lớn	7
2.3 Hàm phi Euler $\phi(n)$ (Euler's Totient Function)	7
2.4. Thuật toán Euclid mở rộng tìm nghịch đảo mô-đun	8
3. Thuật toán RSA	8
4. Ví dụ số học cụ thể	8
5. Chữ ký số RSA (Digital Signature)	9
C. PHÂN TÍCH KIẾN TRÚC VÀ THIẾT KẾ PHẦN CỨNG HỆ THỐNG RSA	10
1. Phân tích yêu cầu kỹ thuật và Ràng buộc thiết kế	10
2. Lựa chọn giải thuật tối ưu cho phần cứng	10
3. Thiết kế kiến trúc tổng thể (Top-Level Architecture)	10
4. Thiết kế các khối thành phần	11
4.1 Module RAM chứa số nguyên tố (prime_ram_512)	11
4.2 Module Nhân số nguyên lớn (serial_multipler)	11
4.3 Module Sinh khóa bí mật (rsa_key_d_generator)	11
4.4 Module Xử lý đệm Wrapper (rsa_pkcs1_v15_wrapper)	12
4.5 Module Tính toán hằng số Montgomery (Fast_R2_Mod_N)	12
4.6 Module nhân Montgomery (MontgomerreyMul)	13
4.7 Module Lũy thừa mô-đun (ModExp)	13
D. TRIỂN KHAI RTL VÀ KẾT QUẢ MÔ PHỎNG, TỔNG HỢP MẠCH	15
1. Môi trường và Công cụ phát triển	16
2. Triển khai mã RTL (RTL Implementation)	16
2.1 Module Prime_ram_512	16
2.2 Module rsa_pkcs1_v15_wrapper	16

2.3 Module serial_multiplier	17
2.4 Module seq_div_unsigned	19
2.5 Module Fast_R2_Mod_N.....	19
2.6 Module rsa_key_d_generator	21
2.7 Module nhân MontgomeryMul.....	23
2.9 Module RSA_Top	30
3. Kết quả mô phỏng chức năng (Functional Simulation).....	37
3.1 Module rsa_pkcs1_v15_wrapper	37
3.2 Module serial_multiplier	38
3.3 Module seq_div_unsigned	38
3.4 Module Fast_R2_Mod_N.....	38
3.5 Module rsa_key_d_generator	38
3.6 Module nhân MontgomeryMul.....	38
3.7 Module ModExp	39
3.8 Module RSA_Top	39
4. Kết quả tổng hợp mạch (Synthesis Results) và Thực thi trên FPGA	40
E. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	42
1. Kết quả đạt được.....	42
2. Hạn chế của thiết kế phần cứng.....	42
3. Hướng phát triển	42
F. TÀI LIỆU THAM KHẢO	43

A. GIỚI THIỆU ĐỀ TÀI:

1. Đặt vấn đề

"Trong kỷ nguyên số hóa và sự phát triển mạnh mẽ của Internet vạn vật (IoT) cùng các hệ thống nhúng, nhu cầu bảo mật thông tin và xác thực dữ liệu trở nên cấp thiết hơn bao giờ hết. Hệ mật mã bất đối xứng RSA với tính an toàn cao dựa trên bài toán phân tích thừa số nguyên tố lớn đã và đang là nền tảng cho nhiều giao thức bảo mật hiện đại. Tuy nhiên, việc thực thi thuật toán RSA bằng phần mềm trên các bộ vi xử lý thông thường thường gặp hạn chế lớn về tốc độ xử lý do phải thực hiện các phép toán số nguyên lớn (lên đến 1024-bit hoặc 2048-bit), gây thắt nút cổ chai cho hệ thống. Chính vì vậy, việc nghiên cứu và triển khai hệ mã hóa RSA trực tiếp trên cấu trúc phần cứng (như chip FPGA/ASIC) là một giải pháp tối ưu nhằm tăng tốc độ tính toán, tối ưu hóa hiệu năng và đáp ứng thời gian thực cho các hệ thống bảo mật chuyên dụng."

2. Mục tiêu nghiên cứu

Nghiên cứu cơ sở lý thuyết toán học của hệ mật mã bất đối xứng RSA và các giải thuật tối ưu hóa tính toán số nguyên lớn; từ đó thiết kế, mô phỏng và hiện thực hóa thành công lõi phần cứng (IP Core) mã hóa/giải mã RSA có hiệu năng cao, tối ưu về tốc độ và tài nguyên trên nền tảng FPGA. Hệ thống đạt được các yêu cầu về mã hóa theo chuẩn PKCS #1

Về mặt lý thuyết:

- Sơ đồ hóa và làm chủ các phép toán nền tảng của RSA như lũy thừa mô-đun (Modular Exponentiation) và nhân mô-đun (Modular Multiplication) với kích thước dữ liệu lớn (1024-bit).
- Nghiên cứu sâu thuật toán nhân Montgomery (Montgomery Multiplication) để loại bỏ phép chia lấy dư phức tạp, chuyển đổi thành các phép cộng và dịch bit tối ưu cho cấu trúc phần cứng.

Về mặt thiết kế vi kiến trúc (Microarchitecture):

- Thiết kế hoàn chỉnh khối xử lý dữ liệu (Datapath) bao gồm các bộ cộng số nguyên lớn, bộ dịch bit và hệ thống thanh ghi lưu trữ dữ liệu 1024-bit.
- Xây dựng khối điều khiển (Control Unit) sử dụng máy trạng thái FSM (Finite State Machine) để điều phối chính xác luồng dữ liệu giữa các khối chức năng.

Về mặt triển khai và kiểm thử phần cứng:

- Hiện thực hóa toàn bộ kiến trúc đã thiết kế bằng ngôn ngữ mô tả phần cứng (Verilog HDL/SystemVerilog) theo phong cách mã mã hóa có thể tổng hợp được (Synthesizable RTL code).

- Xây dựng môi trường kiểm tra (Testbench) trên phần mềm ModelSim/QuestaSim để mô phỏng, kiểm tra dạng sóng (Waveform) và xác trị tính đúng đắn của chức năng mã hóa/giải mã RSA.
- Thực hiện tổng hợp mạch (Synthesis) và tối ưu hóa thời gian (Timing) bằng công cụ Quartus II / Vivado, nhằm đạt được tần số hoạt động Fmax cao nhất và mức tiêu thụ tài nguyên phần cứng (LUTs, Registers) thấp nhất trên chip FPGA mục tiêu

3. Đối tượng và phạm vi nghiên cứu

3.1. Đối tượng:

- **Hệ mật mã bất đối xứng RSA:** Trọng tâm là cấu trúc toán học của thuật toán bao gồm quá trình tạo khóa, mã hóa và giải mã dữ liệu số nguyên lớn.
- **Giải thuật tối ưu hóa phần cứng:** Thuật toán nhân Montgomery (Montgomery Multiplication) và thuật toán lũy thừa nhanh (Square-and-Multiply) áp dụng cho các phép toán mô-đun kích thước lớn.
- **Kiến trúc phần cứng (Microarchitecture):** Cấu trúc của khối xử lý dữ liệu (Datapath), khối điều khiển FSM (Finite State Machine), và các kỹ thuật tối ưu hóa phần cứng (như pipeline, chia khối dữ liệu) để tăng hiệu năng xử lý.
- **Ngôn ngữ và Công cụ thiết kế vi mạch:** Ngôn ngữ mô tả phần cứng Verilog HDL, công cụ tổng hợp mạch và mô phỏng dạng sóng.

3.2. Phạm vi nghiên cứu

- Độ dài khóa triển khai: Đề án tập trung nghiên cứu và hiện thực hóa hệ mã hóa RSA với kích thước khóa cụ thể là 1024-bit (hoặc bạn có thể sửa thành 2048-bit tùy theo cấu hình thực tế đề án của bạn).
- Giới hạn về chức năng phần cứng: Lõi IP phần cứng tập trung vào 2 chức năng chính là Mã hóa (Encryption) và Giải mã (Decryption) dữ liệu đầu vào. *(Lưu ý: Quá trình tạo khóa - Key Generation - do tính phức tạp của việc tìm số nguyên tố lớn ngẫu nhiên nên thường được thực hiện trước ở môi trường ngoài và nạp vào phần cứng dưới dạng tham số, nếu đề án của bạn làm vậy thì nên ghi rõ ở đây để tránh bị bắt bẻ).*
- Môi trường hiện thực và kiểm thử:
 - o Hệ thống được mô tả hoàn toàn bằng ngôn ngữ Verilog HDL chuẩn có thể tổng hợp được (Synthesizable).
 - o Quá trình kiểm tra tính đúng đắn chức năng (Functional Verification) được thực hiện thông qua mô phỏng dạng sóng trên phần mềm ModelSim / QuestaSim.

o Quá trình đánh giá tài nguyên và thời gian (Timing) được thực hiện bằng công cụ Quartus II (hoặc Vivado) trên dòng chip FPGA mục tiêu

4. Cấu trúc đề án

- **Chương 1: Mở đầu** Trình bày lý do lựa chọn đề tài, mục tiêu nghiên cứu tổng quát và cụ thể, đối tượng cũng như phạm vi giới hạn của đề tài thiết kế phần cứng hệ mã hóa RSA.
- **Chương 2: Cơ sở lý thuyết và Thuật toán RSA** Tập trung vào cơ sở toán học của hệ mật mã bất đối xứng, chi tiết các bước trong thuật toán RSA (tạo khóa, mã hóa, giải mã). Đặc biệt, chương này sẽ đi sâu phân tích thuật toán nhân Montgomery (Montgomery Multiplication) – giải thuật nền tảng để tối ưu hóa các phép toán số nguyên lớn trên phần cứng.
- **Chương 3: Kiến trúc và Thiết kế phần cứng hệ thống RSA** Mô tả chi tiết giải pháp kiến trúc vi mạch cho hệ thống. Chương này trình bày sơ đồ khối tổng thể (Top-Level) và đi sâu vào thiết kế chi tiết của hai khối chức năng cốt lõi: Khối xử lý dữ liệu (Datapath – gồm bộ nhân Montgomery và thanh ghi 1024-bit) và Khối điều khiển.
- **Chương 4: Triển khai RTL và Kết quả mô phỏng, tổng hợp mạch** Trình bày quá trình hiện thực hóa thiết kế bằng ngôn ngữ mô tả phần cứng Verilog HDL. Tiếp theo là phân tích kết quả kiểm thử chức năng (Functional Verification) thông qua giản đồ xung (Waveform) trên phần mềm mô phỏng, cùng các số liệu thống kê về tiêu thụ tài nguyên (LUTs, Registers) và phân tích thời gian (Timing Analysis) sau khi tổng hợp mạch bằng công cụ chuyên dụng trên chip FPGA mục tiêu.
- **Chương 5: Kết luận và Hướng phát triển** Đánh giá lại các kết quả đã đạt được của lõi IP RSA so với mục tiêu ban đầu, chỉ ra những điểm hạn chế còn tồn tại trong thiết kế phần cứng và đề xuất các giải pháp nâng cấp, tối ưu hóa kiến trúc hoặc tích hợp hệ thống System-on-Chip (SoC) trong tương lai.

B. CƠ SỞ LÝ THUYẾT VÀ THUẬT TOÁN RSA

1. Tổng quan về mã hóa bất đối xứng

Mỗi thực thể (người dùng, thiết bị hoặc một IP Core phần cứng) trong hệ mật mã bất đối xứng sẽ sở hữu một cặp khóa duy nhất bao gồm: **Khóa công khai (Public Key)** và **Khóa bí mật (Private Key)**.

Khóa công khai (Public Key): Là khóa được công khai rộng rãi cho toàn bộ mạng lưới hoặc lưu trữ trên các máy chủ quản lý chứng thực. Bất kỳ ai cũng có thể tiếp cận và sử dụng khóa này để mã hóa thông tin, hoặc dùng nó để kiểm tra chữ ký số.

Khóa bí mật (Private Key): Là khóa phải được giữ bí mật tuyệt đối và chỉ duy nhất chủ sở hữu biết. Trong các hệ thống phần cứng chuyên dụng, khóa bí mật thường được lưu trữ an toàn bên trong các vùng nhớ bảo mật (Secure Storage) hoặc thanh ghi nội bộ của chip để tránh bị rò rỉ. Khóa bí mật dùng để giải mã dữ liệu đã được mã hóa bằng khóa công khai tương ứng, hoặc dùng để tạo ra chữ ký số.

2. Cơ sở toán học của RSA

Hệ mật mã RSA dựa trên các nguyên lý số luận và đại số trừu tượng. Đối với kiến trúc phần cứng của đề án này, hệ thống sẽ sử dụng một bể chứa số nguyên tố (Prime Pool) trong bộ nhớ để lấy ra hai số nguyên tố p, q kích thước **512-bit**, sau đó toàn bộ quá trình tính toán khóa 1024-bit và mã hóa/giải mã sẽ được hiện thực bằng mạch logic trên chip.

2.1 Phép toán số học mô-đun và lũy thừa hàm mũ (Modular Arithmetic)

Cho số nguyên a và số nguyên dương n , phép toán $a \bmod n$ trả về số dư của phép chia a cho n . Nếu hai số nguyên a và b có cùng số dư khi chia cho n , ta nói a đồng dư với b theo mô-đun n :

$$a \equiv b \pmod{n}$$

Trong thiết kế phần cứng, để xử lý các số nguyên lớn mà không làm tràn các hàng đợi thanh ghi (Registers), ta áp dụng tính chất phân phối của phép toán mô-đun:

$$(a+b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$$

$$(a \times b) \bmod n = ((a \bmod n) \times (b \bmod n)) \bmod n$$

Phép toán lũy thừa mô-đun (Modular Exponentiation):

- Là hạt nhân tính toán của cả quá trình mã hóa ($C = M^e \pmod{N}$) và giải mã ($M = C^d \pmod{n}$).
- Do cỡ dữ liệu lên tới 1024-bit, mạch Datapath không thể tính toán trực tiếp M^e rồi mới chia lấy dư.
- Thay vào đó, kiến trúc phần cứng sẽ triển khai giải thuật Square-and-Multiply (Bình phương và Nhân) phối hợp với Nhân Montgomery để thực hiện đan xen phép nhân và phép giảm mô-đun (Modular Reduction), giữ cho kích thước dữ liệu luôn giới hạn trong vòng 1024-bit ở mọi chu kỳ xung nhịp.

2.2 Tập hợp số nguyên tố và bài toán nhân số nguyên lớn

Thay vì tích hợp một mạch thử nghiệm số nguyên tố Miller-Rabin phức tạp và tốn tài nguyên diện tích chip, hệ thống tối ưu bằng cách lưu trữ một tập hợp các số nguyên tố p và q kích thước 512-bit ngẫu nhiên có sẵn trong Block RAM (BRAM).

Khi kích hoạt pha tạo khóa, hai số p và q được truy xuất từ RAM. Khối tính toán phần cứng sẽ thực hiện phép nhân hai số nguyên lớn 512-bit để tạo ra cấu trúc khóa công khai n :

$$n = p \times q$$

2.3 Hàm phi Euler $\phi(n)$ (Euler's Totient Function)

Hàm phi Euler $\phi(n)$ định nghĩa là số lượng các số nguyên dương nhỏ hơn hoặc bằng n và nguyên tố cùng nhau với n . Vì p và q là hai số nguyên tố phân biệt, theo tính chất nhân tính của hàm Euler, ta có công thức xác định $\phi(n)$ trên phần cứng thông qua các mạch trừ và mạch nhân số nguyên lớn:

$$\phi(n) = (p - 1)(q - 1)$$

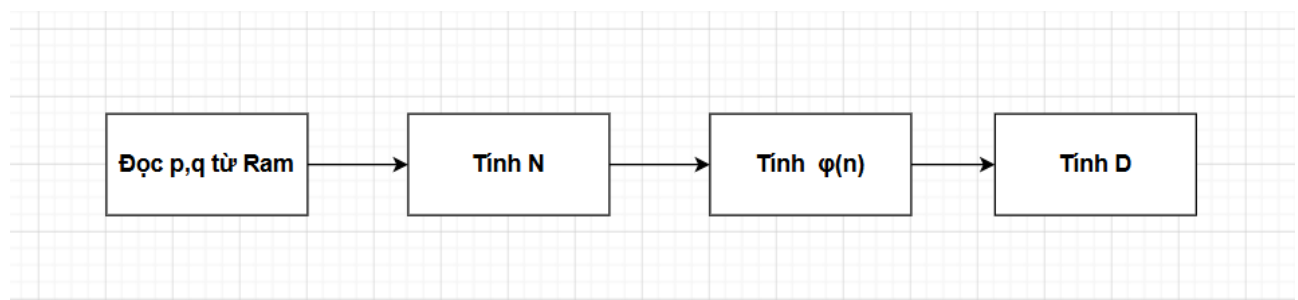
Giá trị $\phi(n)$ sau khi tính toán xong sẽ được lưu vào một thanh ghi tạm thời bên trong chip, giữ bí mật tuyệt đối để phục vụ riêng cho việc tính toán khóa bí mật d , sau đó thanh ghi này sẽ được xóa để đảm bảo an toàn an ninh phần cứng.

2.4. Thuật toán Euclid mở rộng tìm nghịch đảo mô-đun

Sau khi có $\phi(n)$, phần cứng cần xác định cặp khóa chứa thành phần công khai e và thành phần bí mật d .

- Chọn số nguyên dương e sao cho $1 < e < \phi(n)$ và $\gcd(e, \phi(n)) = 1$. (Trong thực tế thiết kế phần cứng, e thường được chọn cố định là một số nguyên tố số học có cấu trúc nhị phân tối ưu như $e = 65537$ nhằm đơn giản hóa tối đa cấu trúc mạch mã hóa).
- Khóa bí mật d là nghịch đảo mô-đun của e theo mô-đun $\phi(n)$, thỏa mãn phương trình đồng dư:

$$exd \equiv 1 \pmod{\phi(n)}$$



3. Thuật toán RSA

- **Bước 1: Tạo khóa (Key Generation):** Chọn p, q , tính $N, \phi(N), D$
- **Bước 2: Mã hóa (Encryption):** Công thức $C = M^e \pmod{N}$.
- **Bước 3: Giải mã (Decryption):** Công thức $M = C^d \pmod{N}$.

4. Ví dụ số học cụ thể

Giả sử hệ thống lấy hai số nguyên tố nhỏ từ RAM là $p = 61$ và $q = 53$.

- **Các thông số khóa được tạo ra:**
 - Mô-đun chung: $n = 3233$
 - Khóa công khai: $e = 17$
 - Khóa bí mật: $d = 2753$
- **Quá trình mã hóa:**

- Dữ liệu đầu vào (bản rõ): $m = 65$
- Công thức thực thi: $C = M^e \bmod N = 65^{17} \bmod 3233$
- Kết quả bản mã ngõ ra: **$C = 2790$**
- **Quá trình giải mã:**
 - Dữ liệu đầu vào (bản mã): $C = 2790$
 - Công thức thực thi: $M = C^d \bmod N = 2790^{2753} \bmod 3233$
 - Kết quả bản rõ khôi phục: **$M = 65$**

5. Chữ ký số RSA (Digital Signature)

Hệ thống phần cứng thực hiện ký và xác thực số bằng cách đảo ngược vai trò của cặp khóa đối với mã băm H của dữ liệu:

- **Quá trình ký số:** Người gửi sử dụng **khóa bí mật d** để tạo ra chữ ký số S :

$$S = H^d \bmod N$$

- **Quá trình xác thực:** Người nhận sử dụng **khóa công khai e** để khôi phục lại mã băm gốc H' :

$$H' = S^e \bmod N$$

Nếu mạch so sánh kết luận $H' = H$, dữ liệu được xác thực an toàn và toàn vẹn.

C. PHÂN TÍCH KIẾN TRÚC VÀ THIẾT KẾ PHẦN CỨNG HỆ THỐNG RSA

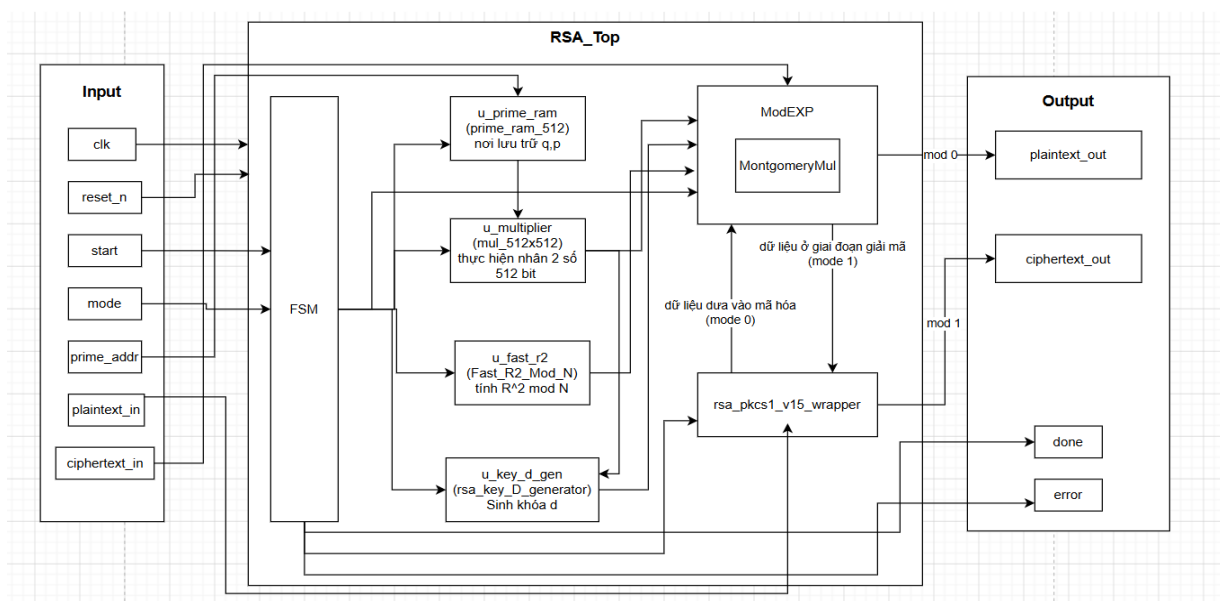
1. Phân tích yêu cầu kỹ thuật và Ràng buộc thiết kế

- Yêu cầu về độ dài khóa (ví dụ: Hệ thống xử lý khối dữ liệu khóa 1024-bit).
- Ràng buộc về tài nguyên phần cứng (Diện tích/Số lượng LUTs trên FPGA) và Tần số hoạt động (Clock frequency).

2. Lựa chọn giải thuật tối ưu cho phần cứng

- **Hạn chế của phép chia lấy dư thông thường:** Phép toán mô-đun thông thường $X \bmod N$ yêu cầu mạch phần cứng phải thực hiện các phép toán chia số nguyên lớn phức tạp, ước lượng thương số và thực hiện chuỗi phép trừ liên tiếp. Điều này tiêu tốn lượng lớn tài nguyên LUTs, kéo dài đường đi tới hạn và làm sụt giảm nghiêm trọng tần số hoạt động (f_{max}) của chip.
- **Giải thuật lũy thừa nhanh (Square-and-Multiply):** Thay vì nhân liên tiếp e lần, giải thuật này quét qua từng bit của số mũ e (từ trái sang phải hoặc từ phải sang trái). Số lượng phép nhân mô-đun giảm từ bậc tuyến tính $O(e)$ xuống bậc logarithmic $O(\log_2 e)$, tương đương tối đa khoảng 1.5 chu kỳ nhân cho mỗi bit của khóa 1024-bit.
- **Giải thuật Nhân Montgomery (Montgomery Multiplication):** Là hạt nhân để tối ưu hóa phần cứng. Giải thuật này chuyển đổi các số nguyên sang miền Montgomery để thực hiện phép tính $A \times B \times R^{-1} \bmod n$. Qua đó, phép chia lấy dư phức tạp được thay thế hoàn toàn bằng các phép cộng số nguyên lớn và **phép dịch phải nhị phân (Shift-right)**, vốn là những phép toán cực kỳ tối ưu và tiết kiệm tài nguyên trên cấu trúc FPGA.

3. Thiết kế kiến trúc tổng thể (Top-Level Architecture)



4. Thiết kế các khối thành phần

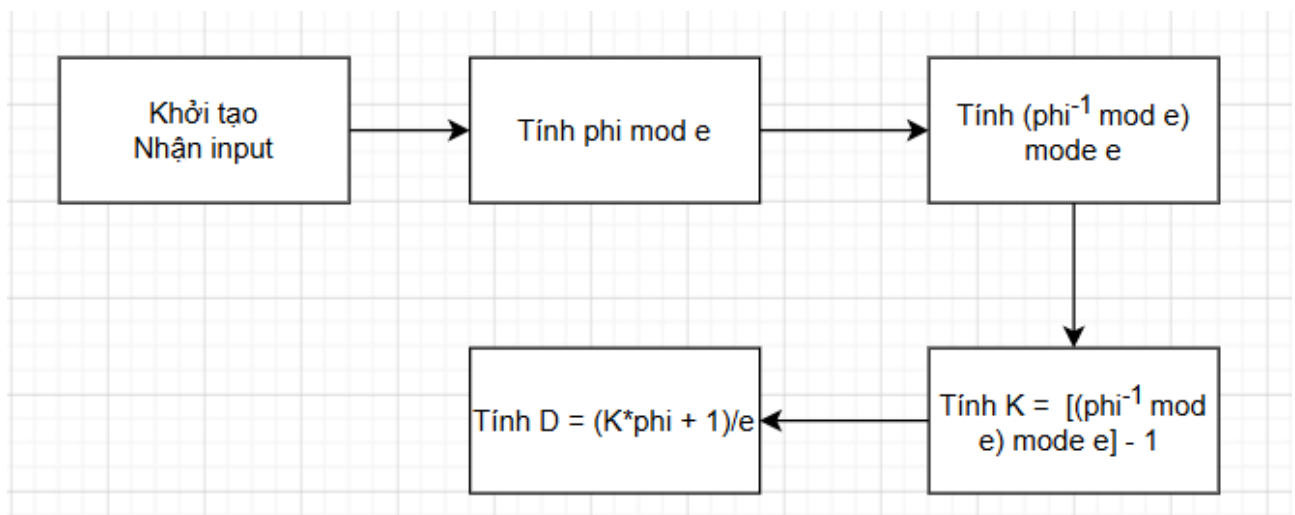
4.1 Module RAM chứa số nguyên tố (prime_ram_512)

- Bản chất phần cứng: Đây là một cấu trúc bộ nhớ dạng Single-Port ROM/RAM được đồng bộ theo xung nhịp hệ thống, không sử dụng máy trạng thái (FSM) để tối ưu tài nguyên.
- Cơ chế vận hành: Khối hoạt động dựa trên cơ chế tra bảng (Lookup Table). Khi nhận địa chỉ **ram_addr** (7-bit) từ FSM tổng, bộ giải mã địa chỉ nội bộ của RAM sẽ trở đến vùng nhớ tương ứng. Sau đúng 1 chu kỳ xung nhịp (**clk**), dữ liệu số nguyên tố 512-bit sẽ xuất hiện ổn định tại ngõ ra **ram_data_out**.

4.2 Module Nhân số nguyên lớn (serial_multipler)

- Sử dụng bộ nhân phân đoạn cơ số lớn. Kiến trúc này giúp hoàn thành phép nhân lớn nhưng vẫn không làm giảm Fmax quá nhiều.
- Nguyên lý luồng dữ liệu:** Thực hiện nhân như một bài nhân tuần tự (dịch phải và quyết định có cộng hay không qua bit cuối của số nhân) khi thực hiện bài toán cộng thực hiện tách nhỏ ra và cộng từng phần thay vì cộng nguyên cả 512-bit sẽ tạo ra một mạch cộng rất lớn.

4.3 Module Sinh khóa bí mật (rsa_key_d_generator)



- Thực hiện phép toán tìm số nghịch đảo mô-đun phục vụ pha tạo khóa, được điều khiển bằng FSM tuần tự.

Giải thuật cơ bản: Hiện thực Thuật toán Euclid mở rộng (Extended Euclidean Algorithm) áp dụng cho số nguyên lớn. Thuật toán giải phương trình đồng dư $ed \equiv 1 \bmod \phi(n)$

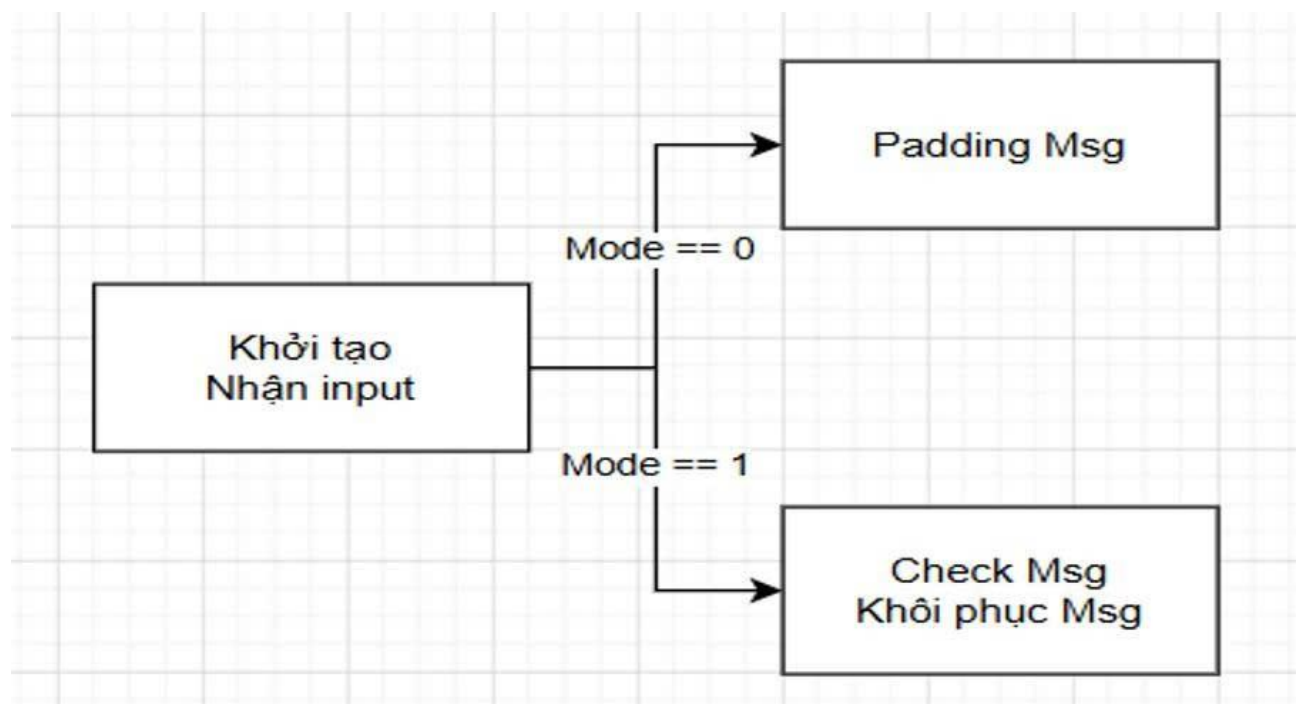
- bằng cách thực hiện liên tiếp các phép chia lấy dư (hoặc phép trừ đại số tuần tự trên phần cứng) để cập nhật liên tục các hệ số Bézout cho đến khi tìm được ước số chung lớn nhất bằng 1

4.4 Module Xử lý đệm Wrapper (rsa_pkcs1_v15_wrapper)

Đảm bảo dữ liệu tuân thủ nghiêm ngặt cấu trúc định dạng an ninh phối hợp trực tiếp với thuật toán RSA, điều khiển bằng FSM phân tích chuỗi .

Giải thuật cơ bản: Hiện thực chuẩn mã hóa **PKCS #1 v1.5 Padding**:

- *Giai đoạn Mã hóa (mode = 0):* Định dạng khối dữ liệu 1024-bit theo cấu trúc nhị phân: EM = 0x00 || 0x02 || PS || 0x00 || [cite_start]M. Trong đó \$M\$ là bản rõ **plaintext_in** (936-bit), còn PS là chuỗi gồm các byte đệm nhị phân ngẫu nhiên sinh ra từ bộ sinh số giả ngẫu nhiên (LFSR) và bắt buộc phải khác 0x00.
- *Giai đoạn Giải mã (mode = 1):* Quét khối dữ liệu 1024-bit thu được sau giải mã toán học. Mạch kiểm tra xem 2 byte đầu tiên có chính xác là 0x00 || [cite_start]0x02 hay không, sau đó dò tìm byte tách biệt 0x00 để bóc gỡ toàn bộ chuỗi đệm PS, trả lại đúng dữ liệu gốc ra cổng **plaintext_out** (936-bit)

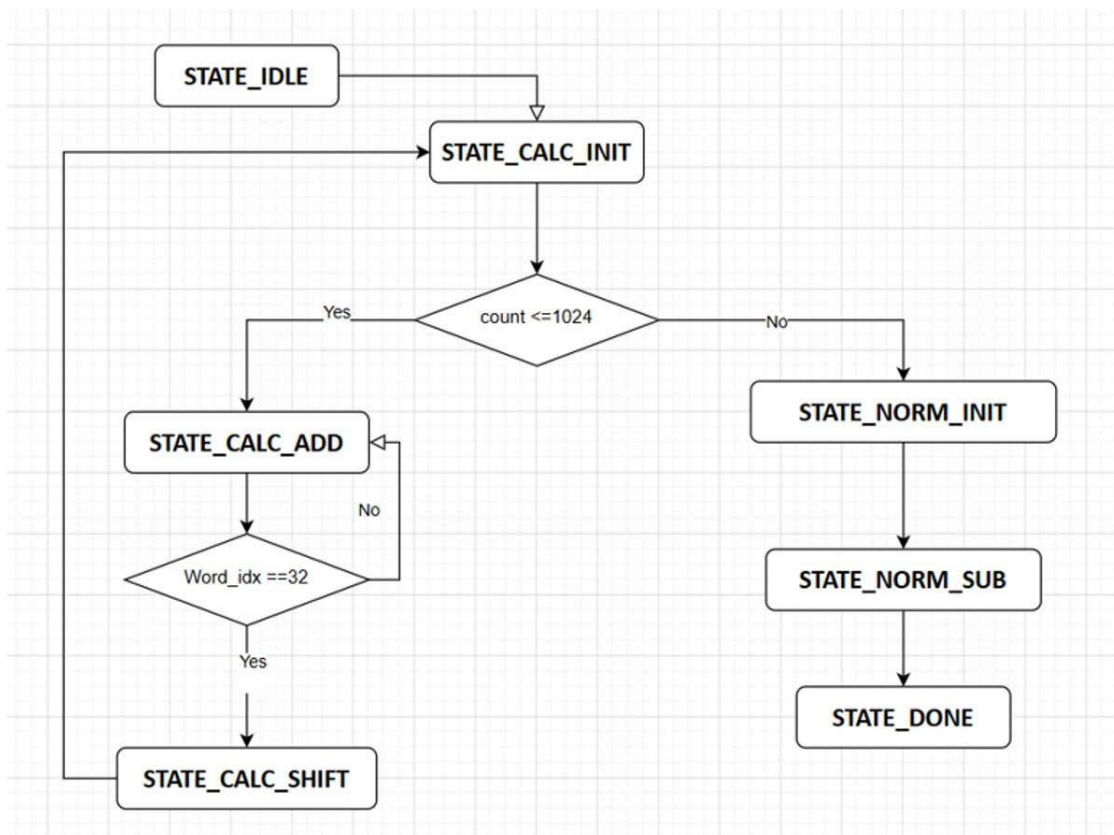


4.5 Module Tính toán hằng số Montgomery (Fast_R2_Mod_N)

- module giúp chuẩn bị hằng số chuyển đổi miền toán học cho hệ thống RSA mà không cần dùng đến mạch chia lớn.
- Giải thuật cơ bản: Thuật toán dịch chuyển và giảm mô-đun liên tiếp (Shift-and-Reduce). Để tính giá trị $R^2 \bmod N$ với $R = 2^{1024}$, mạch khởi tạo một thanh ghi tạm bằng giá trị tích lũy. Vòng lặp được cấu hình chạy đúng 1024 chu kỳ clk. Tại mỗi chu kỳ, mạch thực hiện dịch trái thanh ghi dữ liệu thêm 1 bit (tương đương phép nhân 2), sau đó đưa qua một mạch so sánh tổ hợp: Nếu giá trị sau dịch lớn hơn hoặc bằng mô-đun N, mạch kích hoạt bộ trừ số nguyên lớn để thực hiện phép trừ hiệu chỉnh: $X = X - N$.

4.6 Module nhân Montgomery (MontgomeryMul)

- Lỗi tính toán quyết định tốc độ của toàn bộ kiến trúc phần cứng RSA. Module chạy tuần tự lặp bit điều khiển bằng FSM.
- Giải thuật cơ bản: Giải thuật nhân Montgomery dạng quét từng bit (Radix-2 Montgomery Multiplication). Thuật toán tính toán biểu thức $P = A \times B \times R^{-1} \bmod N$ bằng cách duyệt từ bit thấp nhất đến bit cao nhất của toán hạng A qua 1024 chu kỳ xung nhịp:

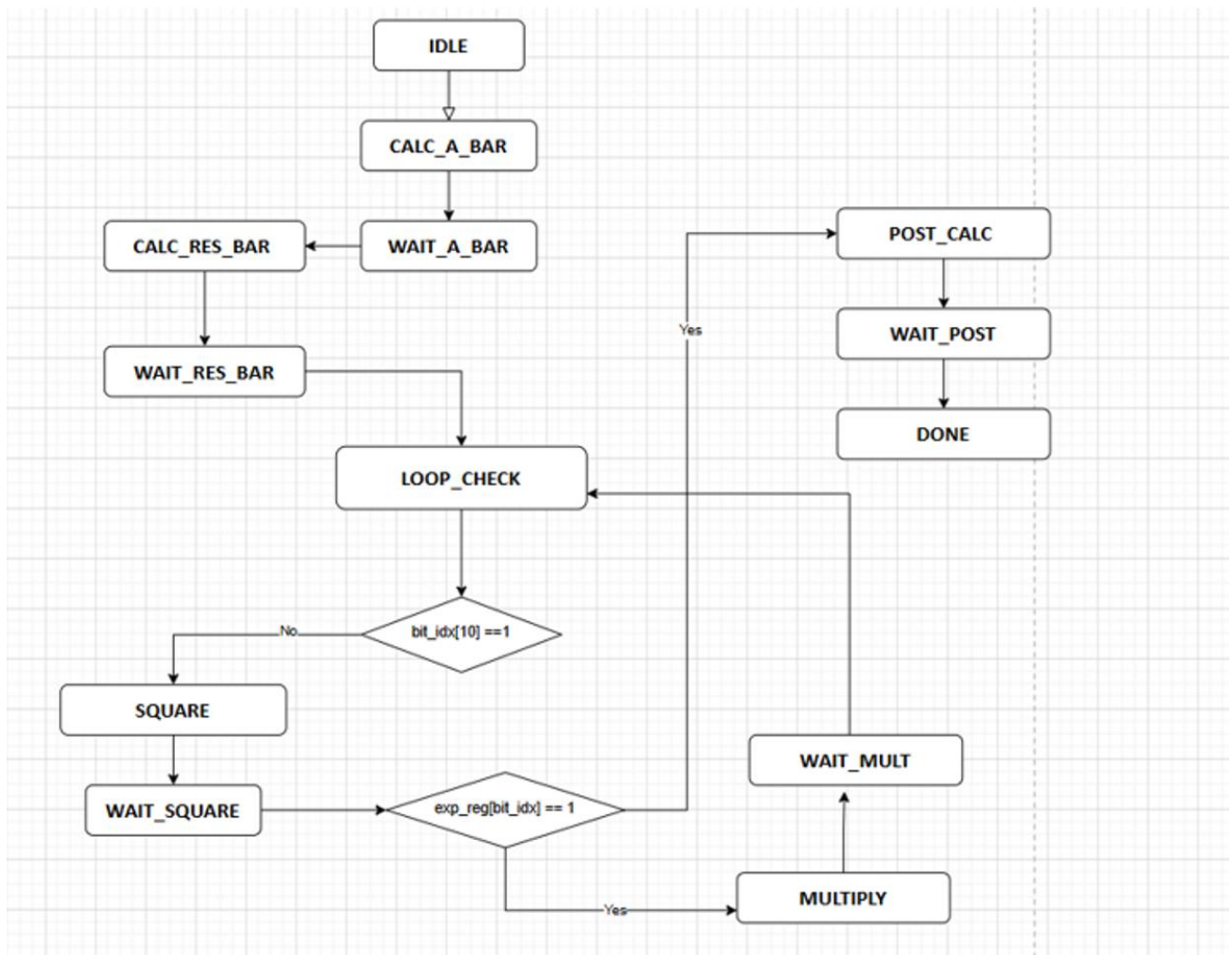


4.7 Module Lũy thừa mô-đun (ModExp)

Tầng thực thi toán học cao nhất của mạch Datapath, quản lý và gọi lắp module **MontgomeryMul** thông qua FSM trung gian.

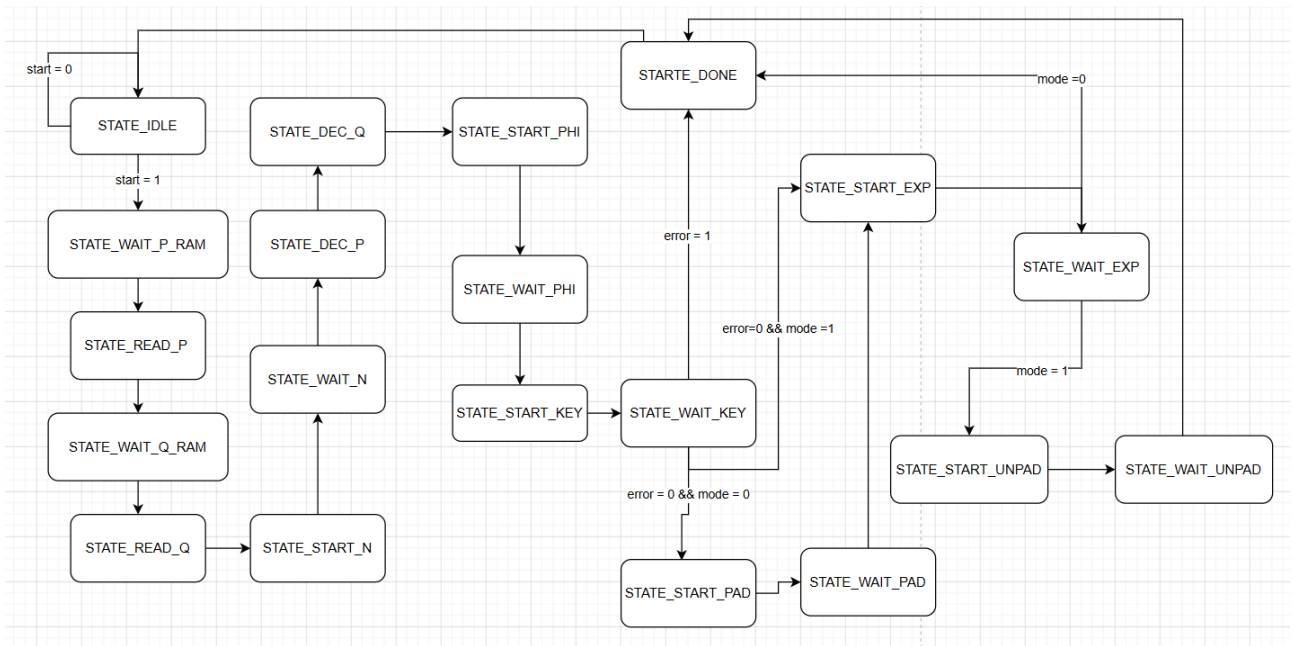
Giải thuật cơ bản: Giải thuật Bình phương và Nhân (Square-and-Multiply) hoạt động hoàn toàn trong miền toán học Montgomery.

- Đầu tiên, mạch chuyển đổi cơ số đầu vào sang miền Montgomery bằng cách gọi bộ nhân tính toán nhân với $R^2 \bmod N$
- Khởi tạo một thanh ghi kết quả tích lũy trong miền Montgomery a
- Bộ điều khiển FSM tiến hành quét từng bit của thanh ghi số mũ **exp_in** (1024-bit):
Tại mọi bit, mạch đều gọi lõi **MontgomeryMul** để tự bình phương chính nó $A = A^2$.
Nếu bit của số mũ đang xét bằng 1, FSM gọi tiếp bộ nhân một lần nữa để nhân thêm cơ số $A = AX$



4.8 Module điều khiển (RSA_top)

- Đây là bộ điều khiển tối cao (Master Controller / Wrapper), chứa máy trạng thái FSM 15 bước chính mà đồ án sử dụng để quản lý, cấu hình và kết nối đồng bộ tất cả các module con nêu trên hoạt động nhịp nhàng theo một chu trình kín .
- **Giải thuật cơ bản:** Sử dụng cơ chế bắt tay tín hiệu phần cứng (Hardware Handshaking). Bộ điều khiển tổng không trực tiếp tham gia tính toán, nó đóng vai trò phân phối dữ liệu thông qua các đường bus nội bộ, cấp phát tín hiệu kích hoạt (**start**) đến từng module chức năng riêng biệt tại từng thời điểm chính xác, sau đó rơi vào trạng thái đứng đợi cho đến khi nhận được cờ báo hoàn thành (**done/ready**) từ module con đó trả về thì mới chuyển trạng thái tiếp theo



D. TRIỂN KHAI RTL VÀ KẾT QUẢ MÔ PHỎNG, TỔNG HỢP MẠCH

1. Môi trường và Công cụ phát triển

- Ngôn ngữ mô tả phần cứng sử dụng Verilog HDL .
- Công cụ phần mềm hỗ trợ: Quartus II / Vivado (Tổng hợp mạch), ModelSim

2. Triển khai mã RTL (RTL Implementation)

2.1 Module Prime_ram_512

Ram chứa 128 số nguyên tố 512 bit khác nhau, truy cập bằng địa chỉ ngẫu nhiên của người dùng để chọn ra cặp số nguyên tố p,q cho hệ thống mã hóa

2.2 Module rsa_pkcs1_v15_wrapper

- Tổng quát
 - Công dụng: Module thực hiện đóng gói là tháo gói thông điệp theo định dạng PKCS#1
 - Nguyên lý hoạt động: Sử dụng cơ chế LFSR để random ra một chuỗi thực hiện làm padding cho tin nhắn gốc điều này sẽ làm cho mỗi phần sinh ra ciphertext của RSA đều không giống nhau đảm bảo hơn về mặt bảo mật. Và thực kiểm tra loại bỏ phần padding để trả lại tin nhắn nguyên vẹn khi giải mã xong
- Chi tiết về module:
 - Cơ chế LFSR: Thực hiện xor 4 bit cuối để lấy LFSR Tap sau đó dịch trái và đưa LFSR Tap vào bit cuối cùng của chuỗi mới (Giả random).

```
wire lfsr_tap = lfsr[63] ^ lfsr[62] ^ lfsr[60] ^ lfsr[59];  
wire [63:0] next_lfsr = {lfsr[62:0], lfsr_tap};
```

- Cơ chế đảm bảo các byte trong chuỗi padding không được = 00: Khi random xuất hiện byte 00 sẽ đổi thành 01.

```
// Loại bỏ các byte có giá trị 0x00 và thay bằng 0x01 để đúng quy định của chuẩn PKCS v1.5  
wire [7:0] rand7 = (lfsr[63:56] == 8'h00) ? 8'h01 : lfsr[63:56];  
wire [7:0] rand6 = (lfsr[55:48] == 8'h00) ? 8'h01 : lfsr[55:48];  
wire [7:0] rand5 = (lfsr[47:40] == 8'h00) ? 8'h01 : lfsr[47:40];  
wire [7:0] rand4 = (lfsr[39:32] == 8'h00) ? 8'h01 : lfsr[39:32];  
wire [7:0] rand3 = (lfsr[31:24] == 8'h00) ? 8'h01 : lfsr[31:24];  
wire [7:0] rand2 = (lfsr[23:16] == 8'h00) ? 8'h01 : lfsr[23:16];  
wire [7:0] rand1 = (lfsr[15:8] == 8'h00) ? 8'h01 : lfsr[15:8];  
wire [7:0] rand0 = (lfsr[7:0] == 8'h00) ? 8'h01 : lfsr[7:0];
```

- Chuỗi padding được sinh ra hoàn tất xử lý random loại bỏ byte 00 và ghép tất cả các byte lại với nhau.

```
// Gop thanh khoi mang 64-bit ngau nhien an toan (Padding String)
wire [63:0] rand_block = {rand7, rand6, rand5, rand4, rand3, rand2, rand1, rand0};
```

- Cấu trúc của một msg đã được padding sẽ có dạng: 0x00 || 0x02 || PS || 0x00 || msg trong đó (0x00 là byte khởi đầu, 0x02 là kiểu padding, PS là chuỗi được sinh ra khi ghép tất cả byte rand ra chuỗi rand_block và 0x00 là byte phân cách giữa padding với phần msg việc này sẽ được thực hiện khi chuẩn bị bắt đầu mã hóa).

```
if (mode == 1'b0) begin
    // CHIEU PADDING (MA HOA): Ghep khoi 1024 bit bat buoc tu chuoai 936 bit dau vao
    // Cau truc: 0x00 (1 byte) + 0x02 (1 byte) + Ngau nhien (8 byte) + 0x00 (1 byte) + Data (117 byte)
    padded_msg_1024 <= {8'h00, 8'h02, rand_block, 8'h00, message_in_936};
    padding_error <= 1'b0;
end
```

- Ở phần khi đã giải mã xong khối sẽ tiến hành kiểm tra tính hợp lệ của file vừa được giải mã qua việc kiểm tra byte bắt đầu byte chế độ padding byte phân cách

```
// CHIEU UNPADDING (GIAI MA): Kiem tra tinh hop le cua cac vi tri byte danh dau co dinh
if (decrypted_msg_1024[1023:1016] == 8'h00 &&
    decrypted_msg_1024[1015:1008] == 8'h02 &&
    decrypted_msg_1024[943:936] == 8'h00) begin

    message_out_936 <= decrypted_msg_1024[935:0]; // Cat lay dung 936 bit thong diep goc ban dau
    padding_error <= 1'b0;
end else begin
    message_out_936 <= 936'd0;
    padding_error <= 1'b1; // Bat co loi neu khoi giai ma bi sai cau truc PKCS v1.5
end
```

2.3 Module serial_multiplier

- Tổng quát
 - Công dụng: Thực hiện phép nhân tuần tự bằng cách tách nhỏ theo CHUNK_WIDTH
 - Nguyên lý: Do tồn tại các phép nhân rất lớn trong thuật toán nên cần tách nhỏ ra từng phần để thực hiện để đảm bảo Fmax không quá thấp làm trì trệ cả hệ thống.
- Chi tiết về Module:
 - Phân tính toán chính: Thực hiện theo quy tắc phép nhân tuần tự dịch bit và cộng vào khi bit bằng 1 đặc biệt ở chỗ phép cộng khi phát hiện bit 1 sẽ được

tách nhỏ 32-bit mỗi lần không cộng luôn 512-bit điều này sẽ tạo ra một mạch cộng lớn ảnh hưởng xấu tới Fmax.

```

CHECK: begin
    if (bit_count == LAST_BIT_COUNT) begin
        chunk_idx <= {CHUNK_IDX_WIDTH{1'b0}};
        state <= PACK;
    end else if (multiplier_reg[0]) begin
        add_carry <= 1'b0;
        chunk_idx <= {CHUNK_IDX_WIDTH{1'b0}};
        state <= ADD_READ;
    end else begin
        shift_carry <= 1'b0;
        chunk_idx <= {CHUNK_IDX_WIDTH{1'b0}};
        state <= SHIFT;
    end
end
end
  
```

```

ADD_READ: begin
    product_read <= product_words[chunk_idx];
    multiplicand_read <= multiplicand_words[chunk_idx];
    state <= ADD_WRITE;
end

ADD_WRITE: begin
    product_words[chunk_idx] <= chunk_sum[CHUNK_WIDTH-1:0];
    add_carry <= chunk_sum[CHUNK_WIDTH];

    if (chunk_idx == LAST_CHUNK_IDX) begin
        shift_carry <= 1'b0;
        chunk_idx <= {CHUNK_IDX_WIDTH{1'b0}};
        state <= SHIFT;
    end else begin
        chunk_idx <= chunk_idx + 1'b1;
        state <= ADD_READ;
    end
end
end
  
```

```

SHIFT: begin
    multiplicand_words[chunk_idx] <= shifted_multiplicand_word;
    shift_carry <= multiplicand_words[chunk_idx][CHUNK_WIDTH-1];

    if (chunk_idx == LAST_CHUNK_IDX) begin
        multiplier_reg <= multiplier_reg >> 1;
        bit_count      <= bit_count + 1'b1;
        chunk_idx      <= {CHUNK_IDX_WIDTH{1'b0}};
        state          <= CHECK;
    end else begin
        chunk_idx <= chunk_idx + 1'b1;
    end
end
end

```

- Sau khi cộng xong thực hiện ghép các word cộng lại để có được kết quả cuối cùng.

```

PACK: begin
    product_out[chunk_idx * CHUNK_WIDTH +: CHUNK_WIDTH] <= product_words[chunk_idx];
    if (chunk_idx == LAST_CHUNK_IDX) begin
        state <= DONE;
    end else begin
        chunk_idx <= chunk_idx + 1'b1;
    end
end
end

```

2.4 Module seq_div_unsigned

- Tổng quát
 - Công dụng: Module thực thi phép chia tuần tự.
 - Nguyên lý: Thực hiện chia tuần tự theo nhiều chu kỳ không để tránh sử dụng trực tiếp phép chia có sẵn trong Verilog làm chậm cả mạch.

2.5 Module Fast_R2_Mod_N

- Tổng quát
 - Công dụng: Module thực thi phép toán $R^2 \bmod N$.
 - Nguyên lý: R^2 là 2^{2028} thay vì tạo 1 số siêu lớn là 2028 ngay từ đầu thì thực hiện tạo ngay remainder = 1 để khởi tạo rồi cứ dịch trái thêm 0 vào 2028 lần là được. Tương tự với hàm Multiplier mấu chốt nằm ở việc phép trừ khi đủ điều kiện (Sau khi dịch trái liên tục R^2 được lưu mảng 32 thanh ghi 32 bit lớn hơn N) thì

sẽ thực hiện phép chia theo từng 32-bit mỗi chu kì tránh để cấu trúc trừ quá lớn được tạo ra làm giảm Fmax

```
SUB_READ: begin
    work_read <= work_words[word_idx];
    n_read    <= n_words[word_idx];
    state     <= SUB_WRITE;
end

SUB_WRITE: begin
    work_words[word_idx] <= sub_word[31:0];
    borrow <= sub_word[32];

    if (word_idx == 6'd32) begin
        count <= count + 1'b1;
        state <= SHIFT_START;
    end else begin
        word_idx <= word_idx + 1'b1;
        state <= SUB_READ;
    end
end
```

2.6 Module *rsa_key_d_generator*

- Tổng quát
 - Công dụng: Dùng để tính toán ra khóa d.
 - Phương pháp: Theo quy trình tính phi mod e và dùng Euclid mở rộng để thực hiện tính toán nghịch đảo kết đó tiếp tục dùng nghịch đảo này để chia dư cho e và tìm bằng cách lấy e trừ cho nghịch đảo của mid mod e chia dư cho e cuối cùng lấy $(k * \text{phi} + 1) / e$ sẽ được d
- Chi tiết về module:
 - State CALC_MOD: Dùng để tính phi mod e.

```

CALC_MOD: begin
    rem <= rem_comp;
    if (bit_cnt == 11'd0) begin
        if (rem_comp == 18'd0) begin
            phi_invalid <= 1'b1;
            private_d   <= 1024'd0;
            state       <= STATE_DONE;
        end else begin
            t           <= 32'sd0;
            new_t       <= 32'sd1;
            r           <= 32'sd65537;
            new_r       <= $signed({15'd0, rem_comp[16:0]});
            inv_cnt     <= 6'd0;
            state       <= CALC_INV;
        end
    end else begin
        phi_mod_shift <= phi_mod_shift << 1;
        bit_cnt <= bit_cnt - 1'b1;
    end
end

```

- State CALC_INV: Tìm nghịch đảo của Phi mod e bằng phương pháp Euclid mở rộng.

```

CALC_INV: begin
    if (new_r != 32'sd0 && inv_cnt < 6'd40) begin
        inv_div_start <= 1'b1;
        state        <= WAIT_INV_DIV;
    end else begin
        if (r > 32'sd1) begin
            phi_invalid <= 1'b1;
            private_d   <= 1024'd0;
            state       <= STATE_DONE;
        end else begin
            state       <= PREP_D_CALC;
        end
    end
end

```

- State PREP_D_CALC: Tính hệ số k

```
PREP_D_CALC: begin
    key_k      <= comb_key_k;
    numerator_shift_reg <= 1041'd0;
    phi_shift_reg <= phi_reg;
    phi_window  <= 17'd0;
    prod_idx    <= 11'd0;
    win_idx     <= 5'd0;
    window_accum <= 6'd0;
    prod_carry  <= 6'd1;
    state       <= GEN_LOAD;
end
```

- State GEN_LOAD: Thực hiện đưa từng bit mới của phi_shift_reg vào

```
GEN_LOAD: begin
    phi_window <= {phi_window[15:0], current_phi_bit};
    if (prod_idx < 11'd1024) begin
        phi_shift_reg <= phi_shift_reg >> 1;
    end
    win_idx     <= 5'd0;
    window_accum <= 6'd0;
    state       <= GEN_SUM;
end
```

- State GEN_SUM: Thực hiện đếm bit từ 0 -> 16 của key_k và phi_window

```
GEN_SUM: begin
    if (key_k[win_idx] && phi_window[win_idx]) begin
        window_accum <= window_accum + 1'b1;
    end

    if (win_idx == 5'd16) begin
        state <= GEN_PRODUCT;
    end else begin
        win_idx <= win_idx + 1'b1;
    end
end
```

- State GEN_PRODUCT dùng tổng vừa tính được để sinh từng bit của key_k * phi + 1.

```
GEN_PRODUCT: begin
    numerator_shift_reg <= {prod_sum[0], numerator_shift_reg[1040:1]};
    prod_carry <= {1'b0, prod_sum[5:1]};

    if (prod_idx == 11'd1040) begin
        private_d <= 1024'd0;
        div_rem <= 18'd0;
        div_idx <= 11'd1040;
        state <= DIVIDE_D;
    end else begin
        prod_idx <= prod_idx + 1'b1;
        state <= GEN_LOAD;
    end
end
```

- State DIVIDE_D thực hiện bước cuối cùng chia cho e để có được khóa private D.

2.7 Module nhân MontgomeryMul

Thực hiện nhân montgomery $Y = AxR^{-1} \bmod N$

```
// --- Khởi logic to hợp xử lý phân đoạn 32-bit mỗi chu kỳ clock ---
wire [31:0] current_s = S[31:0]; // Luôn lấy 32 bit thấp nhất do mạch áp dụng cơ chế xoay vòng

wire [31:0] current_b = a_i ? B_reg[31:0] : 32'd0;
wire [31:0] current_m = q_i ? M_reg[31:0] : 32'd0;
wire [33:0] word_sum = current_s + current_b + current_m + carry; // Cộng to hợp 32-bit kèm cơ nhỡ

// 2. Khởi to hợp cho phép TRỪ (STATE_NORM_SUB)
wire [31:0] norm_m = M_reg[31:0]; // Lấy 32 bit thấp nhất của modulo M đang xoay
wire [32:0] word_sub = current_s - norm_m - borrow; // Trừ to hợp 32-bit kèm cơ mượn
```

- Trích xuất 32 bit thấp nhất của tích lũy S , đem cộng với 32 bit thấp nhất của B (nếu bit quét $a_i = 1$) và 32 bit thấp nhất của M (nếu hệ số hiệu chỉnh $q_i = 1$), kèm theo cờ nhớ carry cũ. Kết quả trả về có độ rộng 34-bit để giữ lại các bit nhớ.
- **Phép trừ (word_sub):** Lấy 32 bit thấp của tích lũy S trừ đi 32 bit thấp của Modulo M và cờ mượn borrow. Kết quả trả về 33-bit dùng để kiểm tra xem giá trị S có vượt quá Modulo hay không


```

STATE_IDLE: begin
    done <= 1'b0; // Ha co bao hoan thanh
    if (start) begin
        // Nap va chen them 32 bit 0 len phia tren cao de dong bo do rong 1056-bit
        A_reg <= {32'd0, a};
        B_reg <= {32'd0, b};
        M_reg <= {32'd0, m};
        S <= 1056'd0; // Xoa thanh ghi tích lũy
        count <= 11'd1; // Dat bo dem vong lap ve 1
        state <= STATE_CALC_INIT;
    end
end

STATE_CALC_INIT: begin
    if (count <= 11'd1024) begin
        word_idx <= 6'd0;
        carry <= 2'd0;
        a_i <= A_reg[0];
        q_i <= S[0] ^ (A_reg[0] & B_reg[0]);
        state <= STATE_CALC_ADD;
    end else begin
        state <= STATE_NORM_INIT;
    end
end
end

```

- Mạch trích xuất bit thấp nhất hiện tại của thanh ghi A ($a_i = A_reg[0]$) và tính toán ngay hệ số giảm cấp Montgomery q_i qua cổng logic XOR, AND. q_i này sẽ cố định suốt quá trình cộng chuỗi 32-bit tiếp theo để đảm bảo bit cuối cùng của kết quả cộng luôn bằng 0 (chia hết cho 2).

```

STATE_CALC_ADD: begin
    // Chen ket qua tinh tong 32-bit vua tinh vao MSB va xoay phai du lieu
    S <= {word_sum[31:0], S[1055:32]};
    B_reg <= {B_reg[31:0], B_reg[1055:32]};
    M_reg <= {M_reg[31:0], M_reg[1055:32]};

    carry <= word_sum[33:32];

    if (word_idx == 6'd32) begin
        // Chay du 33 vong (tu khoi 0 den 32)
        state <= STATE_CALC_SHIFT;
    end else begin
        word_idx <= word_idx + 1'b1;
    end
end

STATE_CALC_SHIFT: begin
    S <= S >> 1;
    A_reg <= A_reg >> 1;
    count <= count + 1'b1;
    state <= STATE_CALC_INIT;
end

STATE_NORM_INIT: begin
    word_idx <= 6'd0;
    borrow <= 1'b0;
    state <= STATE_NORM_SUB;
end
end

```

- 32-bit kết quả vừa tính xong từ khối tổ hợp ($word_sum[31:0]$) được chèn vào các bit cao nhất (MSB) của S , toàn bộ phần dữ liệu cũ của S dịch phải 32 bit

- thực hiện phép **xoay vòng (Rotation)** dữ liệu: Lấy 32 bit thấp nhất đưa ngược lên các bit cao nhất. Việc xoay vòng này giúp đưa phân đoạn 32-bit tiếp theo rơi xuống vị trí thấp nhất để tiếp tục tính toán ở chu kỳ xung nhịp sau, đồng thời sau khi xoay đủ vòng thì dữ liệu gốc vẫn được bảo toàn nguyên vẹn
- vòng lặp con này tăng tiến **word_idx** và chạy liên tục 33 lần (từ khối 0 đến khối 32) để xử lý hết toàn bộ chiều dài thanh ghi 1056-bit. Sau đó chuyển sang trạng thái dịch bit
- Theo thuật toán nhân Montgomery toán học, sau khi kết thúc một lượt cộng tích lũy ứng với một bit của SA , kết quả tạm thời SS phải được chia cho 2, tương đương phép dịch phải 1 bit trong phần cứng ($S \gg 1$)

```
STATE_NORM_INIT: begin
    word_idx <= 6'd0;
    borrow   <= 1'b0;
    state    <= STATE_NORM_SUB;
end

STATE_NORM_SUB: begin
    // Nạp phân đoạn hiệu chỉnh vào MSB và tiến hành xoay vòng các thanh ghi
    S_sub <= {word_sub[31:0], S_sub[1055:32]};
    S     <= {S[31:0],      S[1055:32]};
    M_reg <= {M_reg[31:0],  M_reg[1055:32]};

    borrow <= word_sub[32];

    if (word_idx == 6'd32) begin
        state <= STATE_DONE;
    end else begin
        word_idx <= word_idx + 1'b1;
    end
end

STATE_DONE: begin
    if (borrow) begin
        result <= S[1023:0]; // Nếu có borrow (S < M) -> Giữ nguyên kết quả S
    end else begin
        result <= S_sub[1023:0]; // Nếu không có borrow (S >= M) -> Lấy kết quả đã trừ S_sub
    end
    done <= 1'b1;
    state <= STATE_IDLE;
end
```

- Sau khi chạy xong 1024 vòng lặp nhân, kết quả trong S có thể nằm $> M$ nên cần thực hiện chuẩn hóa $S-M$ để ra kết quả cuối cùng
- Nếu bit mượn borrow = 1 thì tức là $S < M$ kết cuối cùng là S ngược lại thì kết quả cuối cùng sẽ là $S - 1$

2.8 Module ModExp

thực hiện phép toán Lũy thừa Modulo lớn:

$$Y = A^B \bmod N$$

Thay vì thực hiện các phép chia lấy dư (mod N) trực tiếp vốn cực kỳ tốn tài nguyên phần cứng, module này tích hợp bộ nhân **MontgomeryMul** để chuyển toàn bộ các phép toán phức tạp thành chuỗi các phép cộng và dịch bit tuần tự

```
// --- Các thanh ghi lưu trữ nội bộ ---
reg [1023:0] a_bar;
reg [1023:0] res_bar;
reg [1023:0] exp_reg;
reg [1023:0] mod_reg;
reg [10:0] bit_idx; // Đếm từ 1023 lùi về 0 (dùng 11 bit để chứa giá trị âm khi thoát lặp)

// --- Dây tín hiệu giao tiếp với Module 1 (Montgomery Multiplier) ---
reg mul_start;
reg [1023:0] mul_a;
reg [1023:0] mul_b;
wire [1023:0] mul_result;
wire mul_done;

// Gọi (Instantiate) Module 1 đã thiết kế trước đó
MontgomeryMul u_multiplier (
    .clk (clk),
    .rst_n (rst_n),
    .start (mul_start),
    .a (mul_a),
    .b (mul_b),
    .m (mod_reg), // Module không đổi trong suốt quá trình
    .result(mul_result),
    .done (mul_done)
);
```

- **Module ModExp** đóng vai trò là mạch tổng điều khiển. Nó không tự tính toán phép nhân mà gọi lại module nhân từng khối MontgomeryMul đã thiết kế trước đó để thực hiện
- Các thanh ghi mul_start, mul_a, mul_b được điều khiển bởi FSM của ModExp để cấp dữ liệu và lệnh chạy cho khối nhân.
- Khối nhân sau khi xử lý xong (sau nhiều chu kỳ clock) sẽ trả về kết quả qua mul_result và bật cờ hiệu mul_done lên để báo cho mạch tổng biết

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state      <= IDLE;
        mul_start   <= 1'b0;
        done        <= 1'b0;
        bit_idx     <= 11'd1023; // Bắt đầu quét từ bit cao nhất (MSB)
    end else begin
        case (state)
            IDLE: begin
                done <= 1'b0;
                if (start) begin
                    exp_reg <= exp_in;
                    mod_reg <= mod_in;
                    state  <= CALC_A_BAR;
                end
            end
            // --- 1. Chuyển A vào miền Montgomery ---
            CALC_A_BAR: begin
                mul_a      <= base_in;
                mul_b      <= r2_mod_n;
                mul_start <= 1'b1; // Cấp xung start cho Khối nhân
                state      <= WAIT_A_BAR;
            end
            WAIT_A_BAR: begin
                mul_start <= 1'b0; // Tắt xung start
                if (mul_done) begin
                    a_bar <= mul_result; // Lưu lại A_bar
                    state <= CALC_RES_BAR;
                end
            end
        end case
    end
end

```

- Để tính toán theo thuật toán Montgomery, ta cần chuyển số \$A\$ thông thường sang số BAR_A trong miền Montgomery bằng các nhân A vs $R^2 \bmod N$
- Wait_A_BAR tắt cờ mul_start để mạch nhân montgomey không tiếp tục chạy gây lỗi dữ liệu và đợi tính toán xong A trong vùng montgomery

```

// --- 2. Khởi tạo Result = 1 trong miền Montgomery ---
CALC_RES_BAR: begin
    mul_a      <= 1024'd1;
    mul_b      <= r2_mod_n;
    mul_start <= 1'b1;
    state      <= WAIT_RES_BAR;
end
WAIT_RES_BAR: begin
    mul_start <= 1'b0;
    if (mul_done) begin
        res_bar <= mul_result; // Lưu lại Res_bar (thực chất bằng R mod N)
        bit_idx <= 11'd1023; // Đặt lại bộ đếm để quét bit
        state  <= LOOP_CHECK;
    end
end

```

- Tương tự ta cũng có CALC_BAR và WAIT_RES_BAR, giá trị của res_bar được khởi tạo là 1

```

LOOP_CHECK: begin
    // Nếu bộ đếm cuộn vòng qua 0 (thành số âm/giá trị lớn trên 11 bit)
    // Hoặc đơn giản là if (bit_idx == 11'h7FF)
    if (bit_idx[10] == 1'b1) begin
        state <= POST_CALC; // Đã quét hết bit, thoát vòng lặp
    end else begin
        state <= SQUARE;    // Luôn luôn thực hiện Bình phương
    end
end
end

```

- Trạng thái này kiểm tra xem thuật toán đã quét qua hết tất cả các bit của khóa số mũ chưa.
- Do bộ đếm bit_idx đếm ngược từ 1023 về 0, nên khi nó giảm xuống dưới mức 0, giá trị của nó sẽ cuộn vòng sang số âm (trong hệ nhị phân 11 bit, số -1 được biểu diễn là 11'h7FF, tức là bit cao nhất bit_idx[10] sẽ bật lên bằng 1).
- Nếu bit_idx[10] == 1, mạch hiểu là đã duyệt xong toàn bộ 1024 bit và chuyển sang trạng thái kết thúc POST_CALC. Ngược lại, nếu vẫn còn bit chưa quét, mạch bắt đầu thực hiện bước Bình phương (SQUARE).

```

SQUARE: begin
    mul_a    <= res_bar;
    mul_b    <= res_bar;
    mul_start <= 1'b1;
    state    <= WAIT_SQUARE;
end
WAIT_SQUARE: begin
    mul_start <= 1'b0;
    if (mul_done) begin
        res_bar <= mul_result;
        // Kiểm tra xem bit hiện tại của khóa có bằng 1 không?
        if (exp_reg[bit_idx] == 1'b1) begin
            state <= MULTIPLY; // Bit = 1 -> Thực hiện Nhân
        end else begin
            bit_idx <= bit_idx - 1'b1; // Bit = 0 -> Bỏ qua Nhân, giảm index
            state <= LOOP_CHECK;
        end
    end
end
end
end

```

- Theo thuật toán Square-and-Multiply, ứng với mỗi bit của số mũ, biến tích lũy luôn luôn phải được bình phương lên: $\text{bar_res} = \text{Mont}(\text{bar_res}, \text{bar_res})$.
- Tại SQUARE: Mạch đưa giá trị res_bar vào cả hai ngõ đầu vào a và b của bộ nhân để thực hiện phép bình phương trong miền Montgomery.
- Tại WAIT_SQUARE: Sau khi phép bình phương hoàn thành ($\text{mul_done} = 1$), kết quả mới được cập nhật lại vào res_bar. Ngay lập tức, mạch thực hiện kiểm tra giá trị của bit số mũ tại vị trí hiện tại ($\text{exp_reg}[\text{bit_idx}]$):
 - Nếu bit đó bằng 1: Mạch cần làm thêm bước nhân, chuyển tiếp sang trạng thái MULTIPLY.

- Nếu bit đó bằng 0: Không cần làm phép nhân. Mạch thực hiện giảm chỉ số bit ($\text{bit_idx} \leq \text{bit_idx} - 1$) và quay lại LOOP_CHECK để xét bit tiếp theo.

```

MULTIPLY: begin
    mul_a    <= res_bar;
    mul_b    <= a_bar;
    mul_start <= 1'b1;
    state    <= WAIT_MULT;
end
WAIT_MULT: begin
    mul_start <= 1'b0;
    if (mul_done) begin
        res_bar <= mul_result;
        bit_idx <= bit_idx - 1'b1; // Giảm index để xét bit tiếp theo
        state    <= LOOP_CHECK;
    end
end

```

- Trạng thái này chỉ được kích hoạt khi bit của số mũ tại vị trí đang quét có giá trị bằng 1. Phép toán cần thực hiện là nhân giá trị tích lũy với cơ số ban đầu (đều trong miền Montgomery): $\text{bar_res} = \text{Mont}(\text{bar_res}, \text{bar_A})$.
- Mạch truyền res_bar và a_bar vào khối nhân và kích hoạt tính toán.
- Khi có tín hiệu báo xong mul_done, res_bar nhận giá trị tích lũy mới. Mạch tiến hành giảm chỉ số bit bit_idx đi 1 đơn vị để chuẩn bị chuyển sang bit tiếp theo và quay lại trạng thái LOOP_CHECK

```

POST_CALC: begin
    mul_a    <= res_bar;
    mul_b    <= 1024'd1; // Nhân với 1
    mul_start <= 1'b1;
    state    <= WAIT_POST;
end
WAIT_POST: begin
    mul_start <= 1'b0;
    if (mul_done) begin
        result <= mul_result; // Kết quả cuối cùng!
        state    <= DONE;
    end
end
DONE: begin
    done <= 1'b1;
    state <= IDLE; // Quay về chờ chu kỳ tính toán mới
end

```

- Sau khi hoàn thành vòng lặp quét qua tất cả 1024 bit, giá trị nằm trong thanh ghi res_bar vẫn đang ở dạng miền Montgomery. Để đưa nó về lại giá trị số thường chính xác, thuật toán yêu cầu thực hiện phép nhân Montgomery giữa số đó với số 1


```

serial_multiplier #(
    .WIDTH(512),
    .CHUNK_WIDTH(32)
) u_multiplier (
    .clk(clk),
    .rst_n(rst_n),
    .start(mult_start),
    .multiplicand_in(mult_p),
    .multiplier_in(mult_q),
    .product_out(mult_n),
    .done(mult_done)
);

// --- Ket noi Module Fast_R2_Mod_N ---
reg        r2_start;
wire [1023:0] r2_mod_n;
wire        r2_done;

Fast_R2_Mod_N u_fast_r2 (
    .clk(clk), .rst_n(rst_n), .start(r2_start), .n(N_reg), .r2_mod_n(r2_mod_n), .done(r2_done)
);

// --- Ket noi Module rsa_key_d_generator ---
reg        keygen_start;
wire [1023:0] private_d;
wire        keygen_done;
wire        phi_invalid;

```

```

rsa_key_d_generator u_key_d_gen (
    .clk(clk), .rst_n(rst_n), .start(keygen_start), .phi(phi_reg), .private_d(private_d), .done(keygen_done), .phi_invalid(phi_invalid)
);

// --- Ket noi Module ModExp ---
reg        modexp_start;
reg [1023:0] modexp_base;
reg [1023:0] modexp_exp;
wire [1023:0] modexp_result;
wire        modexp_done;

ModExp u_mod_exp (
    .clk(clk), .rst_n(rst_n), .start(modexp_start), .base_in(modexp_base), .exp_in(modexp_exp), .mod_in(N_reg), .r2_mod_n(r2_reg), .result(modexp_result), .done(modexp_done)
);

// --- TICH HOP: Goi Module rsa_pkcs1_v15_wrapper vao mang FSM ---
reg        wrapper_start;
wire        wrapper_ready;
wire        wrapper_padding_error;
wire [1023:0] wrapper_padded_msg;
wire [935:0] wrapper_plaintext_out;

rsa_pkcs1_v15_wrapper u_padding_wrapper (
    .clk(clk),
    .rst_n(rst_n),
    .start(wrapper_start),
    .mode(mode), // Chay chung phuong thuc cau hinh voi FSM Top
    .message_in_936(plaintext_in),
    .message_out_936(wrapper_plaintext_out),
    .padded_msg_1024(wrapper_padded_msg),
    .decrypted_msg_1024(modexp_result), // Lay truc tiep ket qua giai ma tu khoi ModExp de quet
    .ready(wrapper_ready),
    .padding_error(wrapper_padding_error)
);

```


Mạch thực hiện liên kết (instantiate) 6 module con cốt lõi:

- **u_prime_ram**: Bộ nhớ lưu sẵn các số nguyên tố 512-bit an toàn. Chỉ số toán học $addr_q$ tự động tính toán bằng cách tịnh tiến $addr_p + 1$ để lấy ra cặp số (p, q) khác biệt.
- **u_multiplier**: Bộ nhân phần cứng dùng chung, vừa để nhân ra số Modulo N , vừa để tính toán hàm $\phi(N)$
- **u_fast_r2**: Tính toán nhanh hằng số Montgomery $R^2 \bmod N$.
- **u_key_d_gen**: Khối sinh khóa bí mật d dựa trên hàm số Euler $\phi(N)$.
- **u_mod_exp**: Công cụ lũy thừa Modulo lớn (đoạn code mà chúng ta vừa giải thích ở bước trước).
- **u_padding_wrapper**: Bộ tiền xử lý/hậu xử lý đệm bit chuẩn bảo mật PKCS #1 v1.5.

```
case (current_state)

  STATE_IDLE: begin
    done <= 1'b0;
    error <= 1'b0;
    if (start) begin
      ram_addr <= addr_p;
      current_state <= STATE_WAIT_P_RAM;
    end
  end

  STATE_WAIT_P_RAM: begin
    current_state <= STATE_READ_P;
  end

  STATE_READ_P: begin
    p_val <= ram_data_out;
    ram_addr <= addr_q;
    current_state <= STATE_WAIT_Q_RAM;
  end

  STATE_WAIT_Q_RAM: begin
    current_state <= STATE_READ_Q;
  end

  STATE_READ_Q: begin
    q_val <= ram_data_out;
    current_state <= STATE_START_N;
  end

end
```

- Nạp địa chỉ `addr_p` lấy số nguyên tố `p` cất vào `p_val`, ngay sau đó đổi địa chỉ sang `addr_q` lấy số nguyên tố `q` cất vào `q_val`. Sau khi chuẩn bị xong toán hạng, FSM nhảy sang trạng thái tính toán `N`.

```
STATE_START_N: begin
    mult_p      <= p_val;
    mult_q      <= q_val;
    mult_start  <= 1'b1;
    current_state <= STATE_WAIT_N;
end

STATE_WAIT_N: begin
    mult_start <= 1'b0;
    if (mult_done) begin
        N_reg      <= mult_n;
        dec_word_idx <= 5'd0;
        dec_borrow  <= 1'b1;
        current_state <= STATE_DEC_P;
    end
end
```

- **Tính số Modulo N:** Mạch đưa trực tiếp `p` và `q` vào bộ nhân. Khi bộ nhân báo xong (`mult_done`), kết quả 1024-bit được lưu vào `N_reg`.

```
STATE_DEC_P: begin
    p_minus_one[dec_word_idx * 32 +: 32] <= dec_sub_result[31:0];
    if (dec_word_idx == 5'd15) begin
        dec_word_idx <= 5'd0;
        dec_borrow  <= 1'b1;
        current_state <= STATE_DEC_Q;
    end else begin
        dec_word_idx <= dec_word_idx + 5'd1;
        dec_borrow  <= dec_sub_result[32];
    end
end

STATE_DEC_Q: begin
    q_minus_one[dec_word_idx * 32 +: 32] <= dec_sub_result[31:0];
    if (dec_word_idx == 5'd15) begin
        dec_word_idx <= 5'd0;
        dec_borrow  <= 1'b0;
        current_state <= STATE_START_PHI;
    end else begin
        dec_word_idx <= dec_word_idx + 5'd1;
        dec_borrow  <= dec_sub_result[32];
    end
end
```

- áp dụng giải thuật trừ nối tiếp: Mỗi chu kỳ xung nhịp, mạch lấy ra một phân đoạn 32-bit từ vị trí thấp lên vị trí cao thông qua cú pháp toán hạng dải động [dec_word_idx * 32 +: 32]. Phép trừ 32-bit diễn ra rất nhanh, bit mượn dec_sub_result[32] được lưu lại và gởi đầu cho chu kỳ xung nhịp sau.
- Vòng lặp con chạy liên tục 16 lần (dec_word_idx == 15 tức là từ khối 0 đến khối 15, 16x32 = 512 bit) để xử lý hoàn trọn chiều dài của mỗi số nguyên tố, cho ra hai giá trị sạch p_minus_one và q_minus_one

```
STATE_START_PHI: begin
    mult_p      <= p_minus_one;
    mult_q      <= q_minus_one;
    mult_start   <= 1'b1;
    current_state <= STATE_WAIT_PHI;
end

STATE_WAIT_PHI: begin
    mult_start <= 1'b0;
    if (mult_done) begin
        phi_reg      <= mult_n;
        current_state <= STATE_START_KEYS;
    end
end
```

- Sau khi thu được hai kết quả sau phép trừ tuần tự, hệ thống thực hiện tái sử dụng bộ nhân nối tiếp bằng cách nạp p_minus_one và q_minus_one vào ngõ vào đầu vào.
- Khi bộ nhân tính xong tích, kết quả giá trị hàm số Euler $\phi(N)$ độ rộng 1024-bit được lưu vào thanh ghi hệ thống phi_reg và mạch chuyển qua giai đoạn khởi tạo cấu phần khóa.

```

STATE_START_KEYS: begin
    r2_start    <= 1'b1;
    keygen_start <= 1'b1;
    r2_ready    <= 1'b0;
    keygen_ready <= 1'b0;
    current_state <= STATE_WAIT_KEYS;
end

STATE_WAIT_KEYS: begin
    r2_start    <= 1'b0;
    keygen_start <= 1'b0;

    if (r2_done) begin
        r2_reg    <= r2_mod_n;
        r2_ready  <= 1'b1;
    end
    if (keygen_done) begin
        d_reg      <= private_d;
        keygen_ready <= 1'b1;
        if (phi_invalid) begin
            error <= 1'b1;
        end
    end
end

```

```

    if ((r2_ready || r2_done) && (keygen_ready || keygen_done)) begin
        if (error || (phi_invalid && keygen_done))
            current_state <= STATE_DONE;
        else begin
            // Neu la che do Ma hoa (mode=0), phai di qua khoi chen dem truoc
            if (mode == 1'b0)
                current_state <= STATE_START_PAD;
            else
                current_state <= STATE_START_EXP; // Giai ma thi vao thang luon
            end
        end
    end
end

```

Để tăng tốc tối đa, trạng thái STATE_START_KEYS phát lệnh cho hai khối phần cứng độc lập chạy **hoàn toàn song song**: Khối sinh hằng số $R^2 \bmod N$ (r2_start) và Khối sinh khóa bí mật d (keygen_start).

- Bộ điều khiển đứng tại STATE_WAIT_KEYS liên tục giám sát trạng thái của cả hai khối qua các cờ hiệu rẽ nhánh r2_ready và keygen_ready.
- Nếu khối sinh khóa phát hiện tham số ϕ (N) không hợp lệ cho thuật toán RSA (ví dụ không nguyên tố cùng nhau với số mũ công khai e), cờ hiệu phi_invalid bật lên và mạch chuyển thẳng về STATE_DONE kèm theo báo lỗi error = 1.
- Nếu mọi tham số đều chuẩn xác:
 - Chế độ mã hóa (mode == 0): Di chuyển sang trạng thái chèn đệm bảo mật STATE_START_PAD.

- Chế độ giải mã (**mode == 1**): Không cần chèn đệm trước, bỏ qua bước này và nhảy thẳng vào khối lũy thừa modulo **STATE_START_EXP**

```
STATE_START_PAD: begin
    wrapper_start <= 1'b1;
    current_state <= STATE_WAIT_PAD;
end

STATE_WAIT_PAD: begin
    wrapper_start <= 1'b0;
    if (wrapper_ready) begin
        pad_out_reg <= wrapper_padded_msg; // Nhan khoi vector 1024-bit sach sau khi pad
        current_state <= STATE_START_EXP;
    end
end
```

- Chỉ kích hoạt trong chu trình mã hóa. Trạng thái này đánh tín hiệu wrapper_start = 1 ra lệnh cho khối đệm rsa_pkcs1_v15_wrapper làm việc.
- Khối đệm lấy bản rõ tin nhắn 936-bit trộn thêm chuỗi số ngẫu nhiên không chứa giá trị 0 nhằm tạo ra một khối dữ liệu 1024-bit đạt chuẩn mã hóa an toàn. Khi khối này báo xử lý xong (wrapper_ready), dữ liệu đệm được lưu vào thanh ghi pad_out_reg và chuyển sang giai đoạn tính lũy thừa.

```
STATE_START_EXP: begin
    if (mode == 1'b0) begin
        modexp_base <= pad_out_reg; // Ma hoa chuoai da duoc chen dem bao mat
        modexp_exp <= 1024'd65537;
    end else begin
        modexp_base <= ciphertext_in; // Giai ma ban ma 1024-bit truyen tu ngoai vao
        modexp_exp <= d_reg;
    end
    modexp_start <= 1'b1;
    current_state <= STATE_WAIT_EXP;
end

STATE_WAIT_EXP: begin
    modexp_start <= 1'b0;
    if (modexp_done) begin
        if (mode == 1'b0) begin
            ciphertext_out <= modexp_result; // Xuat thang ban ma 1024-bit ra ngoai
            current_state <= STATE_DONE;
        end else begin
            current_state <= STATE_START_UNPAD; // Giai ma xong phai tien hanh unpad
        end
    end
end
```

- **Khi Mã hóa (mode = 0):** Cơ số (modexp_base) là chuỗi đã được chèn đệm bảo mật, số mũ (modexp_exp) được gán cố định bằng số nguyên tố Fermat **65537** (Đây là số mũ công khai tiêu chuẩn mã hóa quốc tế e giúp tối ưu hóa phần cứng). Khi tính xong, bản mã hoàn chỉnh được xuất thẳng ra ngoài qua cổng ciphertext_out.

3.2 Module serial_multiplier

```
VSIM 4> run -all
# === TEST CASE 1 ===
# Ket qua n =
# => SUCCESS: Tich so nho chinh xac!
# === TEST CASE 2 ===
# Ket qua n =
# => SUCCESS: Tich so lon chinh xac!
# ** Note: $finish      : C:/altera/TestRSA/serial_multiplier_tb.v(78)
# Time: 703110 ns Iteration: 0 Instance: /serial_multiplier_tb
```

3.3 Module seq_div_unsigned

```
# === seq_div_unsigned TEST 1 ===
# PASS: 100 / 7 = 14 du 2
# === seq_div_unsigned TEST 2 ===
# PASS: divide_by_zero detected
# ** Note: $finish      : C:/altera/TestRSA/seq_div_unsigned_tb.v(87)
# Time: 990 ns Iteration: 0 Instance: /seq_div_unsigned_tb
```

3.4 Module Fast_R2_Mod_N

```
VSIM 8> run -all
# === Fast_R2_Mod_N TEST ===
# PASS: R^2 mod 3233 =
# ** Note: $finish      : C:/altera/TestRSA/Fast_R2_Mod_N_tb.v(68)
# Time: 5459790 ns Iteration: 0 Instance: /Fast_R2_Mod_N_tb
```

3.5 Module rsa_key_d_generator

```
VSIM 10> run -all
# === TEST CASE 1: Chay voi phi = 67072 ===
# Gia tri private_d thu duoc =
# => SUCCESS: Ket qua tinh d chinh xac tuyet doi!
# === TEST CASE 2: Chay voi phi loi = 65537 ===
# => SUCCESS: Mach da phat hien loi phi_invalid chinh xac!
# === KET THUC QUA TRINH MO PHONG ===
# ** Note: $finish      : C:/altera/TestRSA/rsa_key_d_generator_tb.v(94)
# Time: 464230 ns Iteration: 0 Instance: /rsa_key_d_generator_tb
```

3.6 Module nhân MontgomeryMul

```
# === MontgomeryMul TEST 1 ===
# PASS: Mont(1, R2) = R mod N =
# === MontgomeryMul TEST 2 ===
# PASS: Mont(65, R2) = 65R mod N =
# ** Note: $finish      : C:/altera/TestRSA/MontgomeryMul_tb.v(89)
# Time: 1435350 ns Iteration: 0 Instance: /MontgomeryMul_tb
```


4. Kết quả tổng hợp mạch (Synthesis Results) và Thực thi trên FPGA

- Tài nguyên (Resource Utilization)

Report - RSA_Test_Quartus

Analysis & Synthesis Resource Utilization by Entity									
LC Combinationals	LC Registers	Memory Bits	DSP Elements	DSP 9x9	DSP 18x18	DSP 36x36	Pins	Virtual Pins	
30928 (6971)	38261 (12218)	67616	12	0	6	0	13	3920	RSA_Top

Analysis & Synthesis Resource Usage Summary

	Resource	Usage
1	Virtual pins	3920
2	I/O pins	13
3	Total memory bits	67616
4	DSP block 9-bit elements	12
5	Maximum fan-out node	clk~input
6	Maximum fan-out	38837
7	Total fan-out	247484
8	Average fan-out	3.36

- Tần số hoạt động tối đa (Fmax), timing

Compilation Report - RSA_Test_Quartus

Slow 1200mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	103.04 MHz	103.04 MHz	clk	

This panel reports FMAX for every clock in the design, regardless of the user-specified clock periods. FMAX is only computed for paths where the source and destination are connected to the same clock. Paths of different clocks, including generated clocks, are ignored. For paths between a clock and its inversion, FMAX is computed as FMAX, such that the duty cycle (in terms of a percentage) is maintained. Altera recommends that you always use clock constraints and other slack re

Compilation Report - RSA_Test_Quartus

Table of Contents

- Flow Settings
- Flow Non-Default Global Settings
- Flow Elapsed Time
- Flow OS Summary
- Flow Log
- Analysis & Synthesis
- Fitter
- Flow Messages
- Flow Suppressed Messages
- Assembler
- TimeQuest Timing Analyzer
 - Summary
 - Parallel Compilation
 - SDC File List
 - Clocks
 - Slow 1200mV 85C Model
 - Fmax Summary
 - Timing Closure Recommendation
 - Setup Summary
 - Hold Summary
 - Recovery Summary

Slow 1200mV 85C Model Setup Summary

	Clock	Slack	End Point TNS
1	clk	1.406	0.000

Compilation Report - RSA_Test_Quartus

Table of Contents

- Flow Settings
- Flow Non-Default Global Settings
- Flow Elapsed Time
- Flow OS Summary
- Flow Log
- Analysis & Synthesis
- Fitter
- Flow Messages
- Flow Suppressed Messages
- Assembler
- TimeQuest Timing Analyzer
 - Summary
 - Parallel Compilation
 - SDC File List
 - Clocks
 - Slow 1200mV 85C Model
 - Fmax Summary
 - Timing Closure Recommendation
 - Setup Summary
 - Hold Summary
 - Recovery Summary

Slow 1200mV 85C Model Hold Summary

	Clock	Slack	End Point TNS
1	clk	0.258	0.000

E. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

- Thiết kế hoàn chỉnh các khối tính toán cốt lõi của RSA.
- Mạch mô phỏng đúng chức năng, thời gian mã hóa và giải mã nhanh.

2. Hạn chế của thiết kế phần cứng

- Thiết kế cho ra kết quả mã hóa và giải mã nhanh nhưng tần số lại không cao, gây trở ngại khi tích hợp vào những hệ thống lớn yêu cầu tần số cao.

3. Hướng phát triển

- Áp dụng kiến thức pipeline và chia nhỏ tính toán để có thể giảm chu kỳ và thiết kế cũng có thể chạy được ở tần số cao hơn.

F. TÀI LIỆU THAM KHẢO

- [1] <https://deanqkhanhcoder.github.io/cp-algorithms-vi/algebra/montgomery-multiplication.html>
- [2] <https://ieeexplore.ieee.org/abstract/document/896940>
- [3] https://link.springer.com/chapter/10.1007/3-540-46885-4_24
- [4] [Giải thuật Euclid mở rộng – Wikipedia tiếng Việt](#)
- [5] [RSA Encryption and Decryption Implementation in an FPGA Using Verilog HDL](#)
- [6] [1024-Bit/2048-Bit RSA Implementation on 32-Bit Processor for Public Key Cryptography: IETE Technical Review: Vol 19, No 4](#)
- [7] ["Breaking Weak 1024-bit RSA Keys Using CUDA" by Kerry Scharfglass, Darrin Weng et al.](#)
- [8] https://link.springer.com/chapter/10.1007/978-3-540-40061-5_4